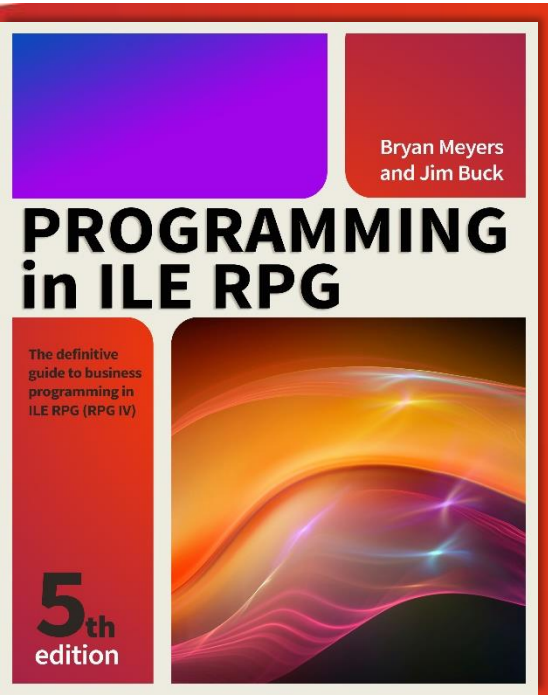
Chapter 3:
Creating and Using Files



Presentation © Copyright 2015, Enskill.com

Mutual information between CIS121 and CIS170

- This chapter references some material that is also covered in CIS121. You will see how the physical and logical file DDS emphasized in CIS121 fits into the RPG programming arena.
- During the latter part of CIS121, you will have an exposure to SQL, and this chapter also covers SQL.
- This “dual coverage” is a good thing because of the relationship between DDS techniques and programming.
- Because some of these techniques are practiced in CIS121, we will focus on the aspects of this chapter that are not covered in CIS121

Concepts/Tasks to focus on



- Continued emphasis on EXTERNAL definition
- SQL Terminology
- Using SQL as a DDL – Data Definition Language - to create files
- SQL Data Types
- Using DDS to define Externally Described Printer Files
  - contains description of printed report appearance

Concepts that will come later / not our focus now

- References to the “RDi” Report Designer – we will learn that later, we are using the traditional PDM/SEU tools for now
- “Navigating Legacy Code” – as before, glance through this section, we will have a learning unit on it at the end of the course

- You will see several items in these slides marked in various ways, such as “more later” or “advanced RPG” or “not in this course” … these items are provided to give you a broad overview of the many techniques

IBM i Database Concepts

- Program described files are created at a record level
  - Not broken down into individual fields
  - RPG program must subdivide record into appropriate fields using fixed format Input or Output specifications (more on this later in the course)
  - NOT an efficient way
- Externally described files break down record to field level (columns)
  - No need for RPG to code field definitions (record layout) within every program
  - Compiler automatically retrieves field definitions into program
  - Record layout is part of file object
  - ***Externally described files are preferred over program described files***

Physical and Logical Files – the focus of CIS121

- Physical files contain data
  - Store records in arrival sequence
  - May designate key field(s)
    - RPG can access records in either key sequence or arrival sequence (first-in, first-out)
    - Unkeyed files are limited to arrival sequence
- Logical files describe how data appears to be stored in physical file
  - Do not actually contain data records
  - Store access paths, or pointers, to records in physical files
  - Logical file based on one or more physical files
  - Provide sorting capability for physical file records
  - Allow selection of certain records from physical file
  - Allow selection of certain fields from physical file record layout

Device Files – CIS170

- Device files associate layout with hardware device
  - E.g., printer or workstation
  - Printer file contains description of printed report appearance (using now)
  - Workstation file describes layout of one or more display screens (more later)
- RPG uses similar coding to process all database files and device files

SQL and DDS

- Structured Query Language (SQL)
  - Industry-standard language for defining, accessing and manipulating database
  - Preferred tool for creating database objects
- Data Description Specifications (DDS)
  - IBM tool
    - Predates SQL
  - Used to:
    - Define physical files (CIS121)
    - Define logical files (CIS121)
    - Define printer files (CIS170 – practiced in this chapter)
    - Define workstation files (CIS170 – later in course)
  - Sole means of creating device files
    - SQL does not address device (printer and workstation) files

Terminology

SQL Term

- Schema (or collection)
- Table
- View
- Index
- Row
- Column

IBM i Term

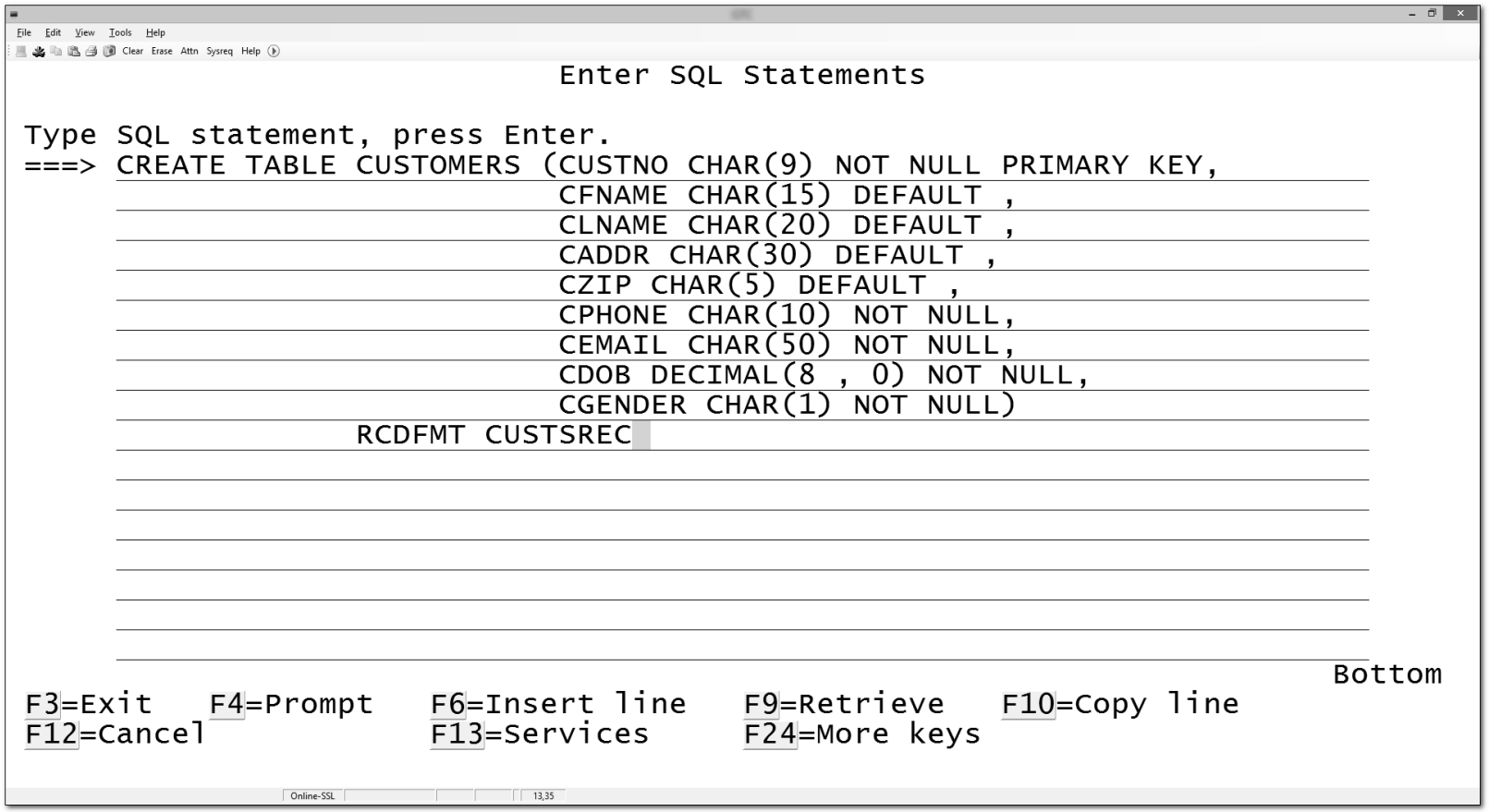
- Library (*LIB)
- Physical file (*FILE PF-DTA)
- Unkeyed logical file (*FILE LF)
- Keyed logical file (*FILE LF)
- Record
- Field

SQL Statement Categories

- Data Definition Language (DDL) (will practice this in this chapter)
  - Used to ***create*** new database objects, and ***alter*** existing objects
  - E.g., CREATE TABLE, CREATE VIEW
- Data Manipulation Language (DML) (practiced in CIS121 later)
  - ***Retrieve*** and ***manipulate*** existing data
  - SELECT, INSERT, UPDATE, DELETE
- Embedded SQL host language statements (COVERED IN ADVANCED RPG)
  - Incorporated into RPG program
  - E.g., DECLARE CURSOR, FETCH
- SQL Procedural Language (SPL) statements (Advanced Database Topic)
  - Used to create SQL programs and functions executed within SQL environment

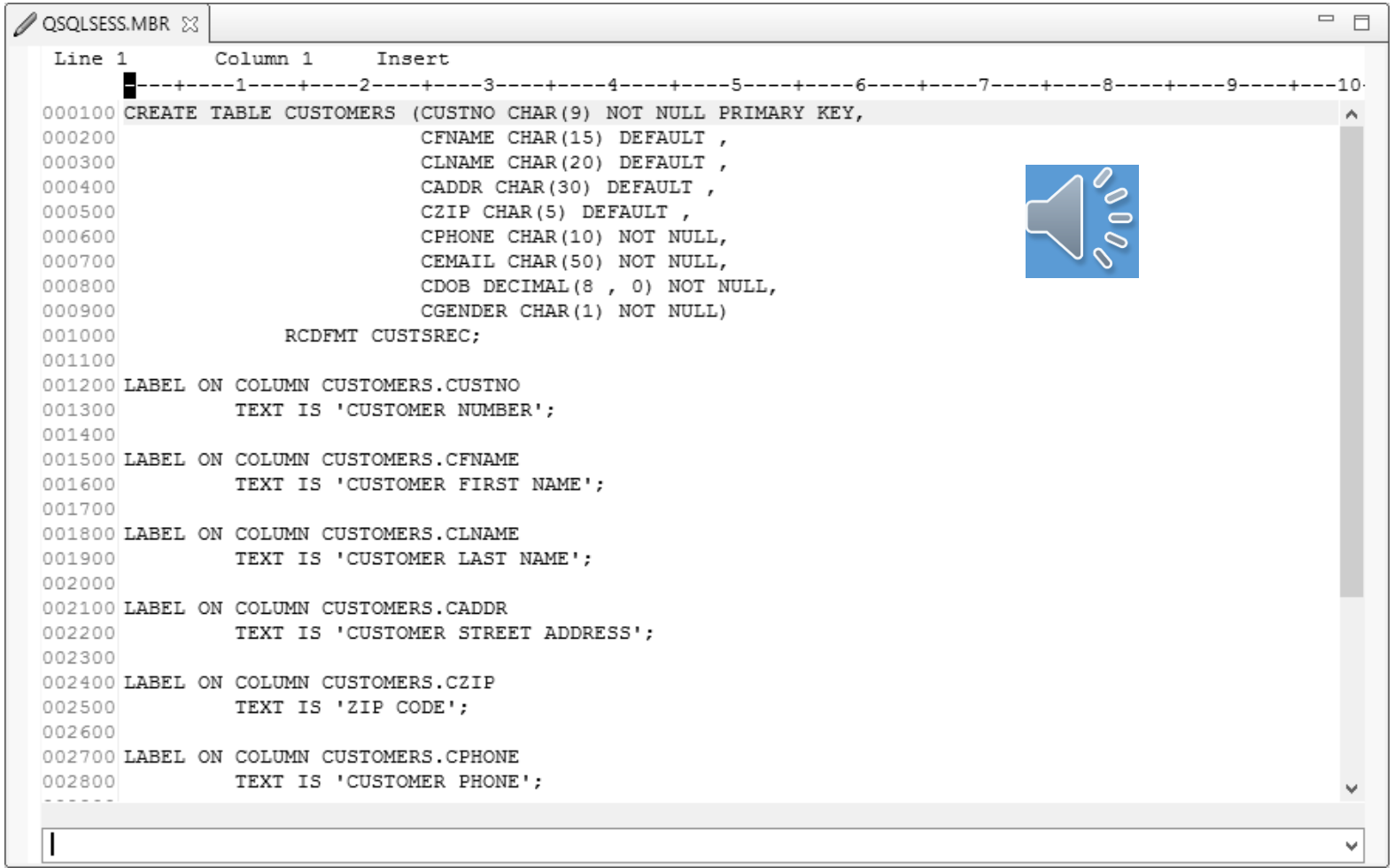
SQL Development Tools

- Interactive SQL – also covered in the latter part of CIS121
  - SQL command line environment
  - `STRSQL` (Start SQL Interactive Session) command



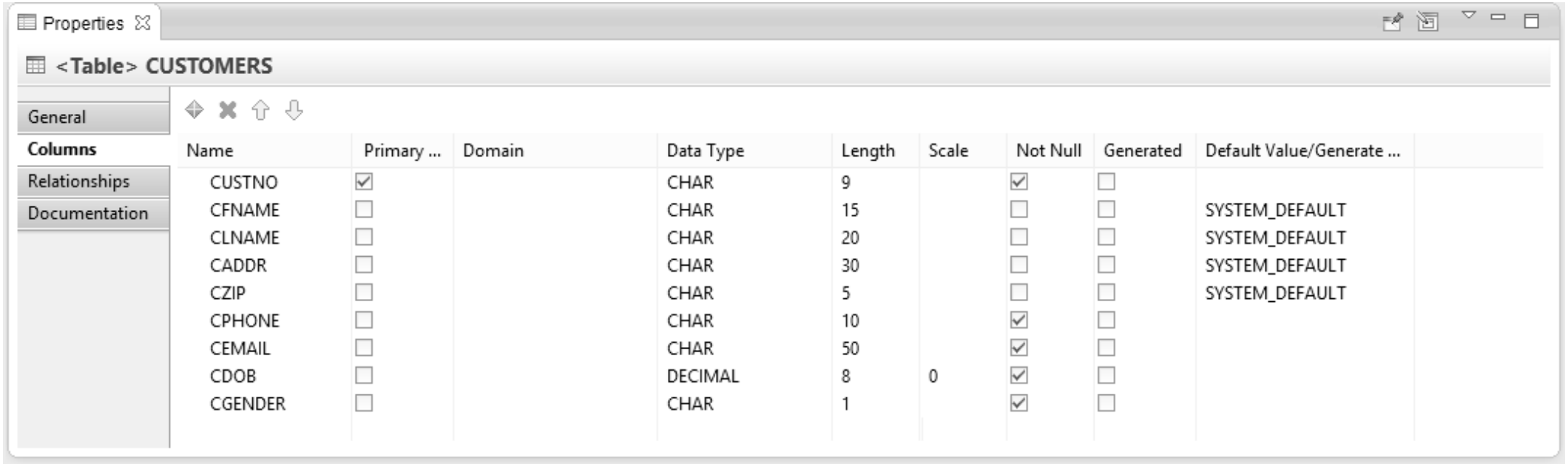
SQL Development Tools

- `RUNSQLSTM` (Run SQL Statements) command
  - Executes SQL script in source file
    - Source is typed into a source member in the same manner as typing in other code



SQL Development Tools

- IBM Data Studio (not covered in this course)
  - Graphical tools for developing and administering databases
  - Can be installed alongside with RDi
    - RDi also includes a Data perspective, with many same capabilities
  - Generates SQL script



Name	Primary ...	Domain	Data Type	Length	Scale	Not Null	Generated	Default Value/Generate ...
CUSTNO	☑		CHAR	9		☑	☐	
CFNAME	☐		CHAR	15		☐	☐	SYSTEM_DEFAULT
CLNAME	☐		CHAR	20		☐	☐	SYSTEM_DEFAULT
CADDR	☐		CHAR	30		☐	☐	SYSTEM_DEFAULT
CZIP	☐		CHAR	5		☐	☐	SYSTEM_DEFAULT
CPHONE	☐		CHAR	10		☑	☐	
CEMAIL	☐		CHAR	50		☑	☐	
CDOB	☐		DECIMAL	8	0	☑	☐	
CGENDER	☐		CHAR	1		☑	☐	

SQL Database Objects

- Table
  - Physical file (*FILE PF-DTA)
  - Contains data records
- View
  - Unkeyed logical file (*FILE LF)
  - Contains alternate access path to table
    - Selects subset of table rows
    - Selects subset of table columns
- Index (advanced database topic)
  - Keyed logical file (*FILE LF)
  - Contains alternate access path to reorder (sort) table rows

Creating Tables with SQL

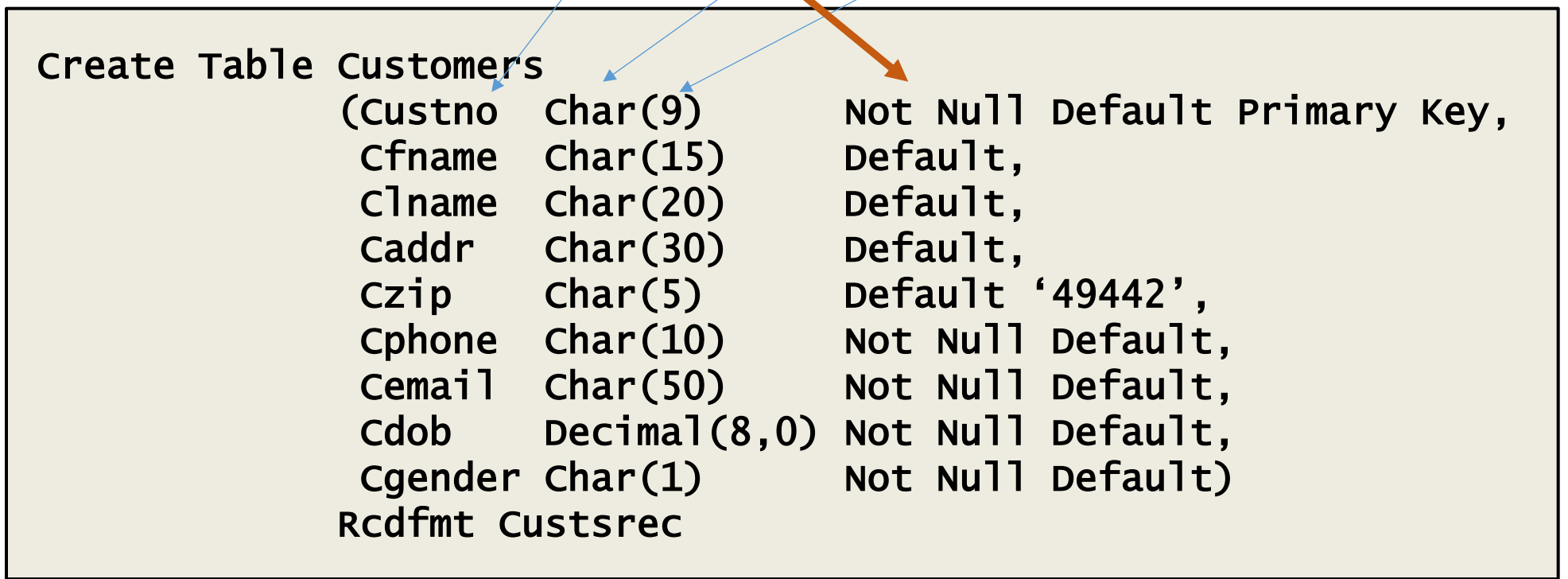
- Table name – keep to 10 characters or less as that is how many characters an IBM i object name can contain
- Column definitions (column name – 10 characters or less for same reason and data type)
- Record format name – required by IBM i (10 characters or less)

```
Create Table Customers
             (Custno  Char(9)       Not Null Default Primary Key,
              Cfname  Char(15)      Default,
              Clname  Char(20)      Default,
              Caddr   Char(30)      Default,
              Czip    Char(5)       Default '49442',
              Cphone  Char(10)      Not Null Default,
              Cemail  Char(50)      Not Null Default,
              Cdob    Decimal(8,0)  Not Null Default,
              Cgender Char(1)       Not Null Default)
             Rcdfmt Custsrec
```



Column Definitions

- Must include at least column name, data type (and size)
- May include optional constraint clauses to further define column



```
Create Table Customers
             (Custno  Char(9)       Not Null Default Primary Key,
              Cfname  Char(15)      Default,
              Clname  Char(20)      Default,
              Caddr   Char(30)      Default,
              Czip    Char(5)       Default '49442',
              Cphone  Char(10)      Not Null Default,
              Cemail  Char(50)      Not Null Default,
              Cdob    Decimal(8,0)  Not Null Default,
              Cgender Char(1)       Not Null Default)
             Rcdfmt Custsrec
```

Common SQL Data Types

- Character
  - CHAR
    - Fixed length character
  - VARCHAR
    - Variable length character
- Date
  - DATE
  - TIME
  - TIMESTAMP
- Numeric
  - DECIMAL
    - Packed numeric
  - NUMERIC
    - Zoned numeric
  - SMALLINT
    - 2-byte integer
  - INTEGER
    - 4-byte integer
  - BIGINT
    - 8-byte integer

Common Column Constraints

- NOT NULL
  - Column cannot contain null value
- DEFAULT
  - Column should contain default value if no value is supplied
- UNIQUE
  - Column value must be unique for every row in table
- PRIMARY KEY
  - Column (or combination of columns) will have unique value to identify specific row

Record Format Name

- Record format (layout) must be named
- RPG requires record format name be distinct (different) from table name
- RCDFMT clause names format

```
Create Table Customers
              (Custno  Char(9)       Not Null Default Primary Key,
               Cfname  Char(15)      Default,
               Clname  Char(20)      Default,
               Caddr   Char(30)      Default,
               Czip    Char(5)       Default '49442',
               Cphone  Char(10)      Not Null Default,
               Cemail  Char(50)      Not Null Default,
               Cdob    Decimal(8,0)  Not Null Default,
               Cgender Char(1)       Not Null Default)
              Rcdfmt Custsrec
```

Qualified Names and Naming Conventions

- May refer to objects by simple name or qualified name
- *Qualified name includes name of schema (library)* in which object is stored
- Naming convention determines qualifier character
  - IBM i System naming convention uses slash (/)
    - E.g., Flightproj/Customers
    - Only supported for IBM i objects
    - Default naming convention
    - `Set Option Naming = *Sys`
  - Industry Standard SQL naming convention uses period (.)
    - E.g., Flightproj.Customers
    - `Set Option Naming = *Sql`



Simple Names and Naming Conventions

- SQL implicitly qualifies simple object names with current schema value
  - Current schema is default location for creating new objects
- Current schema default value depends upon naming convention
  - *SYS naming convention uses job library list (*LIBL) path
  - *SQL naming convention uses user profile name
- Can set current schema with SQL statement
  - E.g., `SET SCHEMA = FLIGHTPROJ`
  - To reset default, `SET SCHEMA = DEFAULT`

Data Storage



- Bit (binary digit) is smallest distinct value
- Combination of four bits represents hexadecimal value
- Byte comprises eight bits
  - Bit pattern in byte represents character
- IBM i uses EBCDIC coding scheme
  - E.g., Bit pattern 11000001 (X'C1') represents 'A'
- Many other systems use ASCII coding
  - E.g., Bit pattern 01000001 (X'41') represents 'A'

Bit Pattern	Hexadecimal Value
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

EBCDIC and ASCII Coding Examples

Character	EBCDIC		ASCII	
	Bits	Hex	Bits	Hex
(space)	0100 0000	40	0010 0000	20
a	1000 0001	81	0110 0001	61
b	1000 0010	82	0110 0010	62
c	1000 0011	83	0110 0011	63
d	1000 0100	84	0110 0100	64
e	1000 0101	85	0110 0101	65
f	1000 0110	86	0110 0110	66
g	1000 0111	87	0110 0111	67
h	1000 1000	88	0110 1000	68
i	1000 1001	89	0110 1001	69
j	1001 0001	91	0110 1010	6A
k	1001 0010	92	0110 1011	6B
l	1001 0011	93	0110 1100	6C
m	1001 0100	94	0110 1101	6D
n	1001 0101	95	0110 1110	6E
o	1001 0110	96	0110 1111	6F
p	1001 0111	97	0111 0000	70
q	1001 1000	98	0111 0001	71
r	1001 1001	99	0111 0010	72
s	1010 0010	A2	0111 0011	73
t	1010 0011	A3	0111 0100	74

Character	EBCDIC		ASCII	
	Bits	Hex	Bits	Hex
u	1010 0100	A4	0111 0101	75
v	1010 0101	A5	0111 0110	76
w	1010 0110	A6	0111 0111	77
x	1010 0111	A7	0111 1000	78
y	1010 1000	A8	0111 1001	79
z	1010 1001	A9	0111 1010	7A
A	1100 0001	C1	0100 0001	41
B	1100 0010	C2	0100 0010	42
C	1100 0011	C3	0100 0011	43
D	1100 0100	C4	0100 0100	44
E	1100 0101	C5	0100 0101	45
F	1100 0110	C6	0100 0110	46
G	1100 0111	C7	0100 0111	47
H	1100 1000	C8	0100 1000	48
I	1100 1001	C9	0100 1001	49
J	1101 0001	D1	0100 1010	4A
K	1101 0010	D2	0100 1011	4B
L	1101 0011	D3	0100 1100	4C
M	1101 0100	D4	0100 1101	4D
N	1101 0101	D5	0100 1110	4E
O	1101 0110	D6	0100 1111	4F

Character	EBCDIC		ASCII	
	Bits	Hex	Bits	Hex
P	1101 0111	D7	0101 0000	50
Q	1101 1000	D8	0101 0001	51
R	1101 1001	D9	0101 0010	52
S	1110 0010	E2	0101 0011	53
T	1110 0011	E3	0101 0100	54
U	1110 0100	E4	0101 0101	55
V	1110 0101	E5	0101 0110	56
W	1110 0110	E6	0101 0111	57
X	1110 0111	E7	0101 1000	58
Y	1110 1000	E8	0101 1001	59
Z	1110 1001	E9	0101 1010	5A
0	1111 0000	F0	0011 0000	30
1	1111 0001	F1	0011 0001	31
2	1111 0010	F2	0011 0010	32
3	1111 0011	F3	0011 0011	33
4	1111 0100	F4	0011 0100	34
5	1111 0101	F5	0011 0101	35
6	1111 0110	F6	0011 0110	36
7	1111 0111	F7	0011 0111	37
8	1111 1000	F8	0011 1000	38
9	1111 1001	F9	0011 1001	39

EBCDIC and ASCII Sequence Comparison

EBCDIC

- Most special characters
- Lowercase alphabet (a-z)
- Uppercase alphabet (A-Z)
- Numbers (0-9)

ASCII

- Many special characters
- Numbers (0-9)
- Uppercase alphabet (A-Z)
- Lowercase alphabet (a-z)

SQL Character Data

- Character data can be any EBCDIC symbol
  - Alphabetic letters, numbers, and special characters (e.g., punctuation marks, currency symbols)
  - Character string is a sequence of characters.
  - Character occupies single byte
- CHAR
  - Fixed length character string
  - All values in column have same length
    - May be padded with blanks
- VARCHAR
  - Variable length character string
  - Column length is uncertain
    - Not padded with blanks
  - SQL specifies maximum length

```
Create Table Customers
      (… ItemDesc Char(35),
         Email    Varchar(256), …)
```

SQL Numeric Data

- Limited to numeric digits 0-9
- SQL defines numeric data with precision and scale
  - Precision … total number of digits available in the number
  - Scale … how many of those digits are to right of decimal point
- Sign is embedded in right-most position
- Thousands separators (commas) and radix character (decimal point) are not stored in the actual value on disk
- Data type determines how variable is represented and stored
  - DECIMAL
  - NUMERIC
  - INTEGER, SMALLINT, BIGINT

SQL Numeric Data

- NUMERIC
  - Zoned (or signed) decimal number
  - ***Each digit occupies full byte***
  - Zone of rightmost digit represents sign

Value to Represent	Bits (3 bytes)	Hex	Characters
+136	11110001 11110011 11110110	F1 F3 F6	136
-136	11110001 11110011 11010110	F1 F3 D6	13O

```
Create Table Customers
      (… CredLimit Numeric(9,2),…)
```

(precision is 9, scale is 2)

SQL Numeric Data

- DECIMAL - preferred, most efficient
  - ***Packed*** decimal number
  - ***Each digit occupies ½ byte*** (4 bits) also known as a "nybble"
  - Sign occupies additional ½ byte

Value to Represent	Bits (2 bytes)	Hex	Characters
+136	00010011 01101111	13 6F	■?
-136	00010011 01101101	13 6D	■_

```
Create Table Customers
      (… Balance Decimal(9,2),…)
```

SQL Numeric Data

- SMALLINT, INTEGER, BIGINT
  - Integers store numeric values in binary format
  - Most compact numeric format
  - Each bit is assigned decimal value, values of “on” bits are added together
  - Three pre-determined sizes, *no decimal places*
  - Programmer does not code the scale or precision, because these are predetermined
    - SMALLINT is 2 bytes, containing 5 digits
    - INTEGER is 4 bytes, containing 10 digits
    - BIGINT is 8 bytes, containing 19 digits

Value to Represent	Bits (2 bytes)	Hex	Characters
+136	0000 0000 1000 1000	00 88	■h
-136	1111 1111 0111 1000	FF 78	■ì

SQL Numeric Data

- SMALLINT, INTEGER, BIGINT determined by scale

Range of allowed values

Data Type	Bytes	Digits	Lowest Value	Highest Value
SMALLINT	2	5	-32,768	32,767
INTEGER	4	10	-2,147,483,648	2,147,483,647
BIGINT	8	19	-9,223,372,036,854,775,808	9,223,372,036,854,775,807

```
Create Table Customers
      (… Terms Integer,…)
```

SQL Date and Time Data Types

- Represent dates and times
  - Neither character nor numeric data types
  - Do not specify scale or precision
- DATE
- TIME
- TIMESTAMP
  - Represents combination of date and time

```
Create Table Customers
  (… Renewal Date,
     Calltime Time,
     Lastupdate Timestamp, …)
```

Creating Views with SQL (database topic)

- View name
- Fullselect
  - May list individual fields – or * for all fields – from table
  - May include Where clause to select specific records
  - May not include Order by clause
- Record format name – optional

```
Create View Custmales
            As Select * From Customers Where Cgender = 'M'
            Rcdfmt Custmrec
```

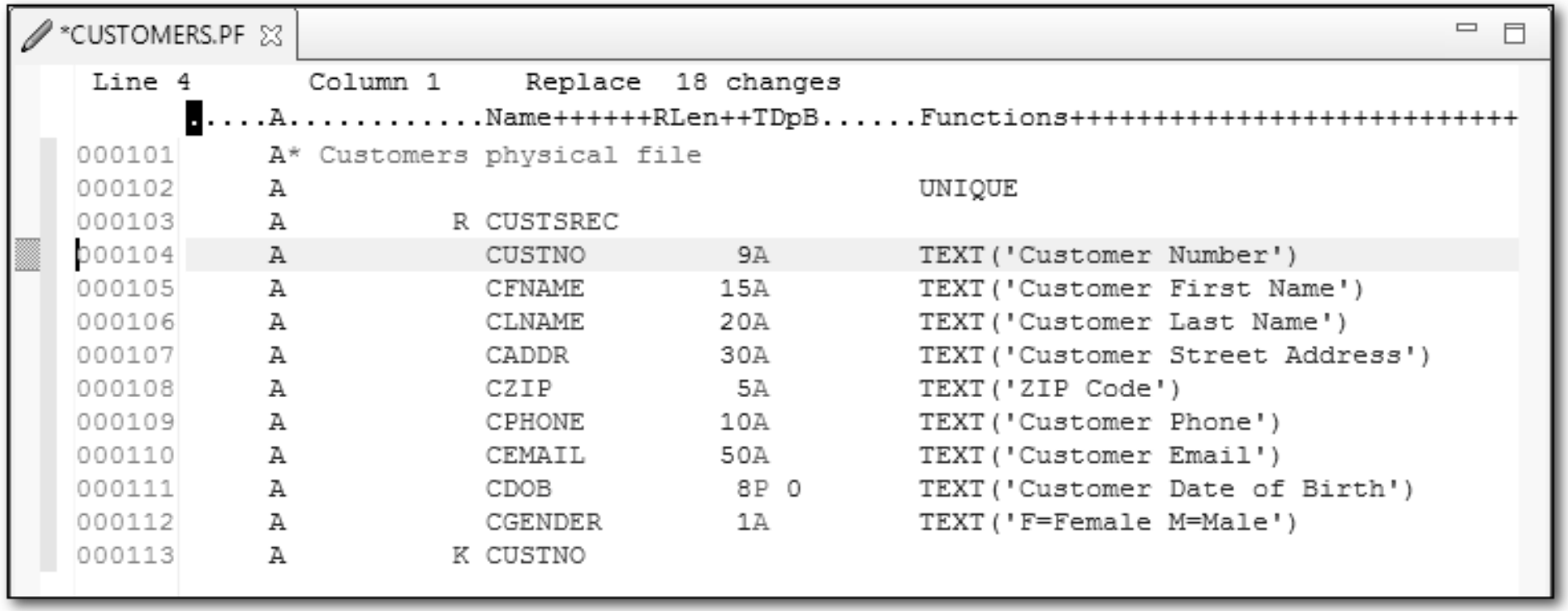
Creating Indexes with SQL (Advanced database topic)

- Index name
- Based on table name
- Where clause – optional
- List of columns in index – key fields
- Record format – optional

```
Create Index Custidx
             On Customers Where Cstate = 'FL' (Czip, Clname)
             Rcdfmt Custxrec
```

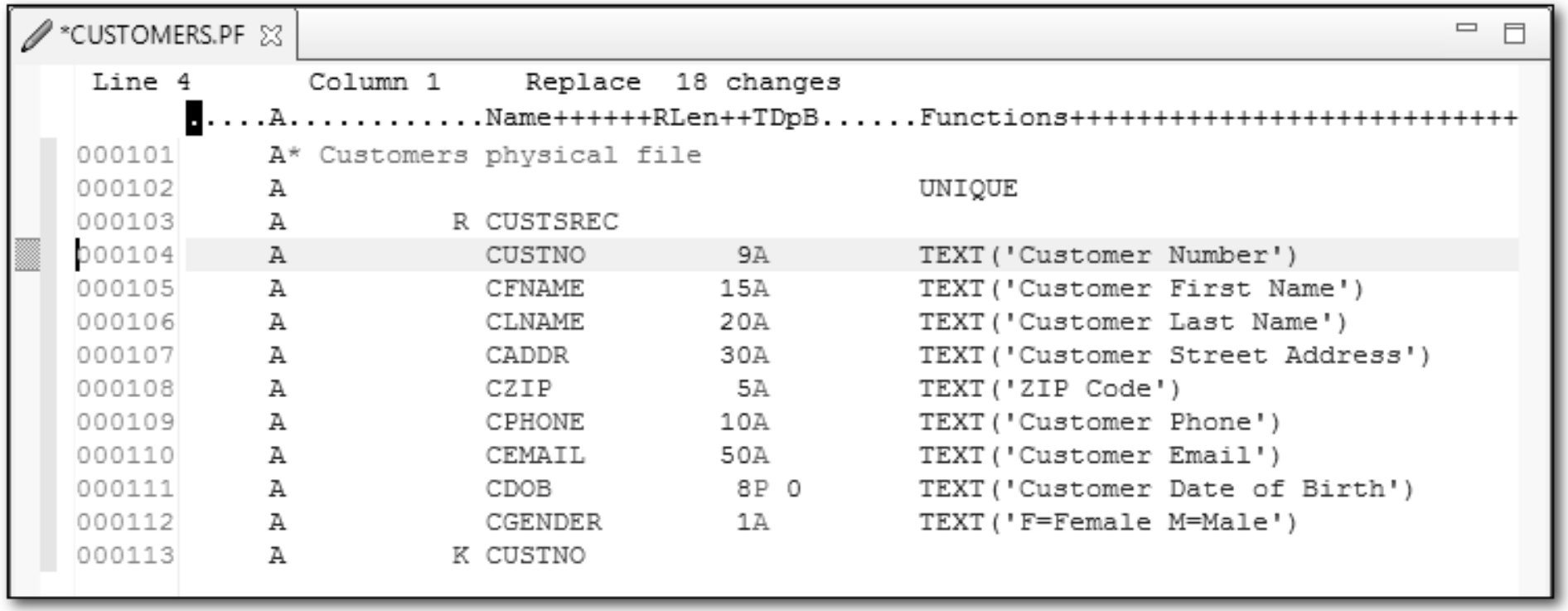
DDS Database Concepts (CIS121)

- Data Description Specifications are stored in source member, then compiled to externally described file object
  - Default source file is QDDSSRC



DDS for Physical Files (CIS121)

- Record format is named using R in position 17
  - Field definitions follow record format
  - Field definitions include data attributes



DDS for Physical Files (CIS121)

- Positions 30-34 contain field length
  - Numeric precision (number of digits)
- Position 35 indicates data type
  - A = Character
    - Default for non-numeric fields
  - P = Packed numeric
    - Default for numeric fields
  - S = Zoned (signed) numeric
  - L = Date
  - T = Time
  - Z = Timestamp
- Positions 36-37 specifies numeric scale (decimal digits)

DDS for Physical Files (CIS121)

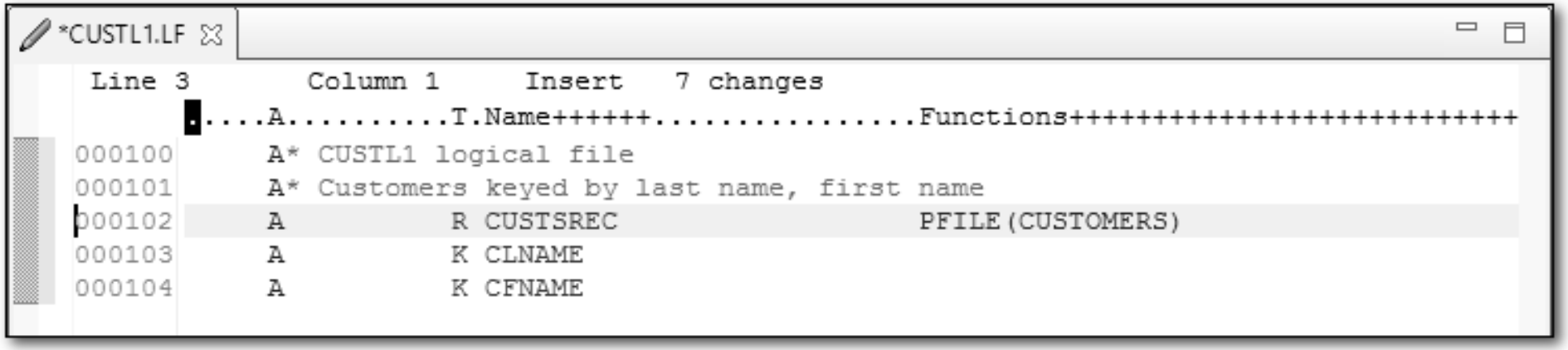
- K in position 17 designates key field(s)
  - List composite key fields from major to minor order
- UNIQUE file level keyword prohibits duplicate key values

*CUSTOMERS.PF

```
 Line 4       Column 1     Replace  18 changes
      .....A.............Name++++++RLen++TDpB......Functions++++++++++++++++++++++++++++
000101      A* Customers physical file
000102      A                                       UNIQUE
000103      A           R CUSTSREC
000104      A             CUSTNO          9A        TEXT('Customer Number')
000105      A             CFNAME         15A        TEXT('Customer First Name')
000106      A             CLNAME         20A        TEXT('Customer Last Name')
000107      A             CADDR          30A        TEXT('Customer Street Address')
000108      A             CZIP            5A        TEXT('ZIP Code')
000109      A             CPHONE         10A        TEXT('Customer Phone')
000110      A             CEMAIL         50A        TEXT('Customer Email')
000111      A             CDOB            8P 0      TEXT('Customer Date of Birth')
000112      A             CGENDER         1A        TEXT('F=Female M=Male')
000113      A           K CUSTNO
```

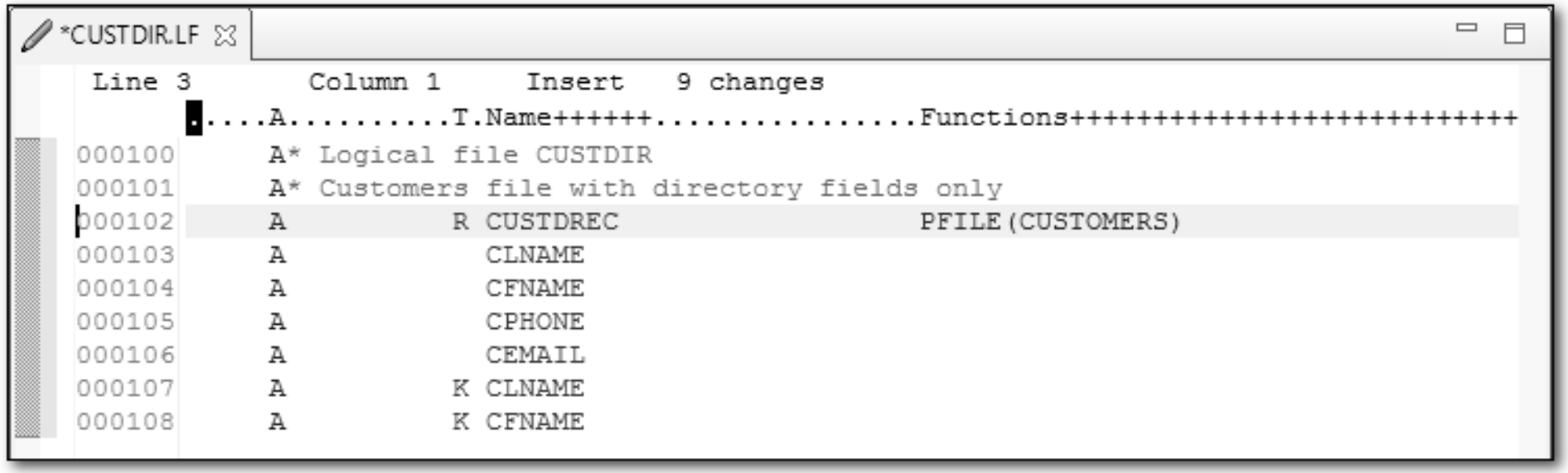
DDS for Logical Files (CIS121)

- PFILE keyword names physical file upon which logical file is based
- Logical file may be used to list alternative key fields
  - Reorder file
  - All fields are included



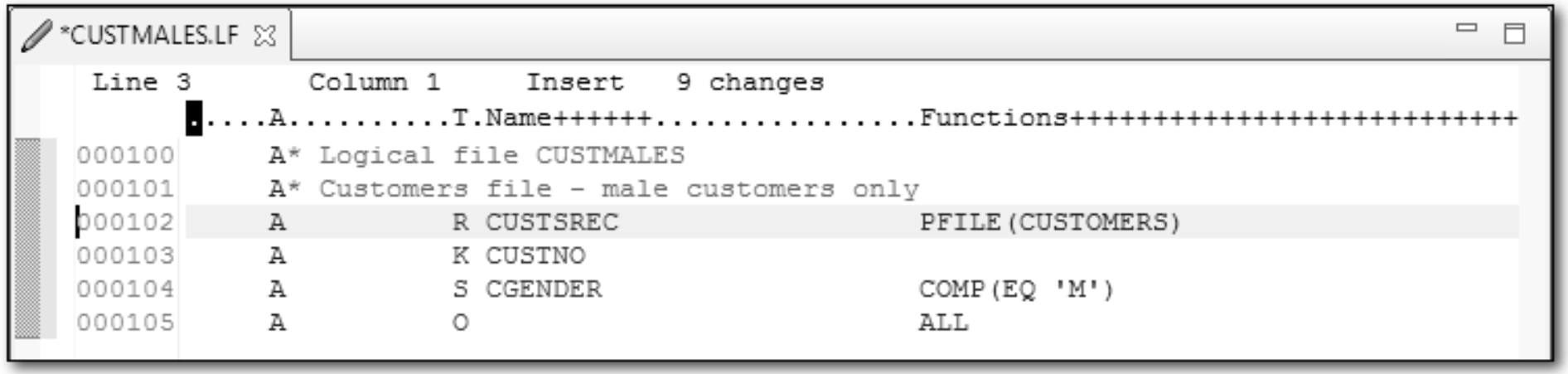
DDS for Logical Files (CIS121)

- Logical file may be used to create new record layout
  - Restrict access to only some of the fields in physical file layout
  - Also known as “projection”



DDS for Logical Files (CIS121)

- Logical file may be used to restrict access to selected records, referred to as a “subset”
  - S in position 17 indicates record selection criteria
  - O in position 17 indicates records to omit



Creating Database Files with DDS (CIS121)

- Physical files
  - Source type PF
  - Compiled using CRTPF command (when using PDM Option 14, this is what occurs)
    - File object attribute is *FILE PF-DTA
    - File contains no data
- Logical files
  - Source type LF
  - Compiled using CRTLF command (when using PDM Option 14, this is what occurs)
    - File object attribute is *FILE LF
  - Physical file must exists before logical file can be created
    - Logical files must be deleted before physical file can be deleted

Changing Physical Files (CIS121)

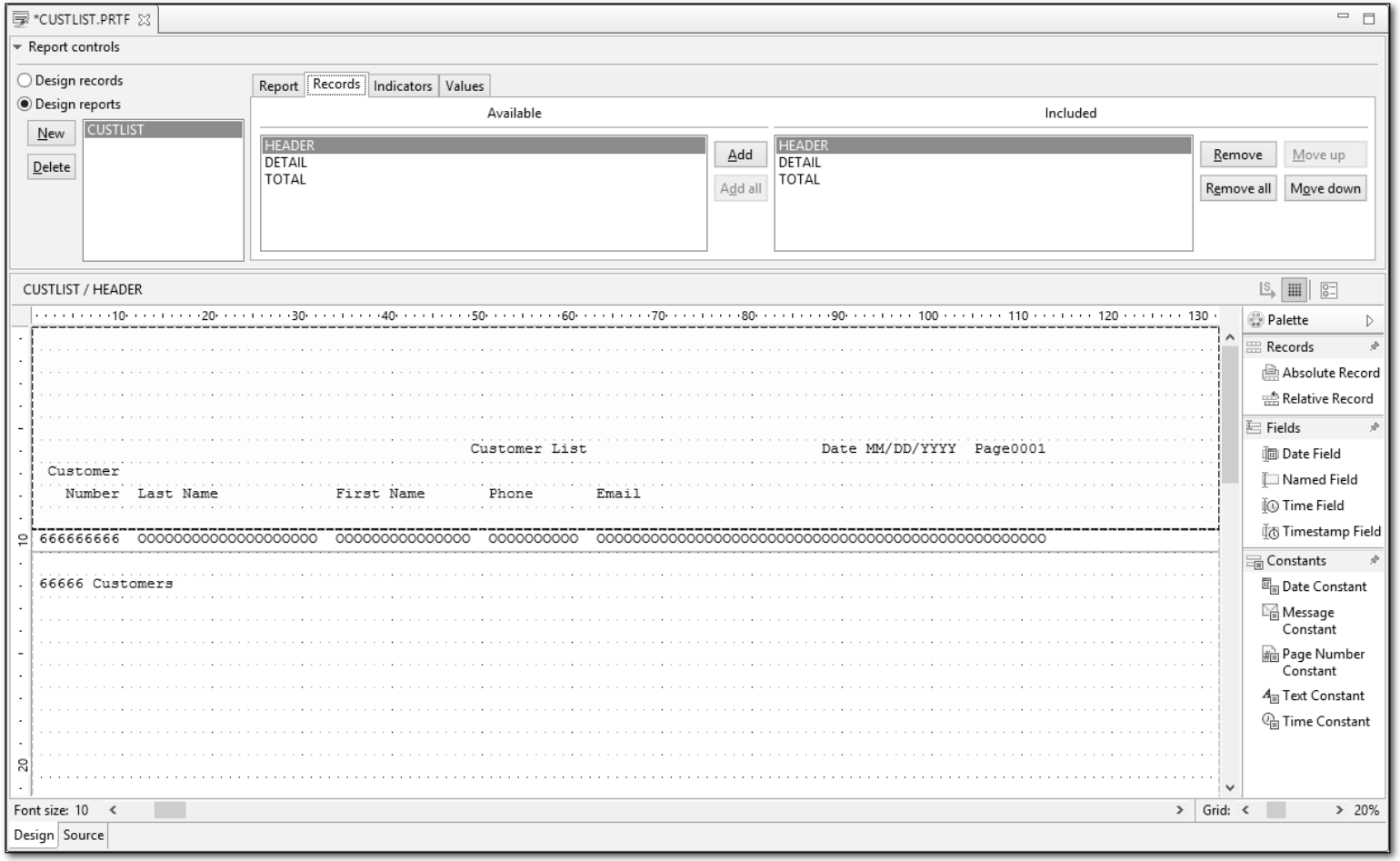
- CHGPF command changes physical file attributes without requiring existing physical or logical files to be deleted
  - Prevents data loss
- Must recompile RPG programs that use changed files
  - Level check prevents running program that uses obsolete database definition

Externally Described Printer Files - Focus

- DDS describes externally described printer files
  - Source member type PRTF
  - Compiled using CRTPRTF command
- Record formats identify one or more printed lines
  - DDS may contain multiple record formats

RDi Report Designer (will be used later)

- Tool enables visual design of printer file

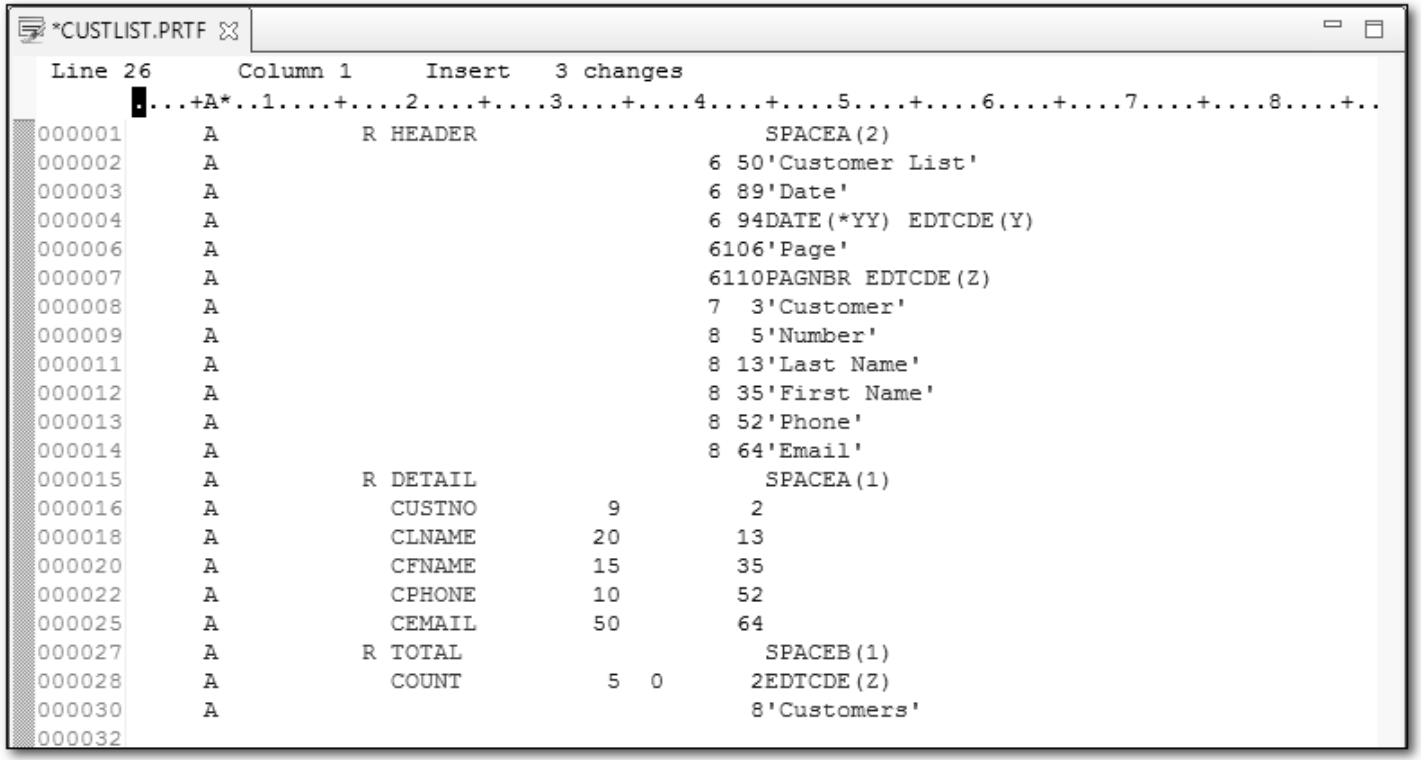


Traditional Printchart

- A “printchart” is a traditional tool used to layout the report’s appearance.
- You will see an example of this in the video in this unit.

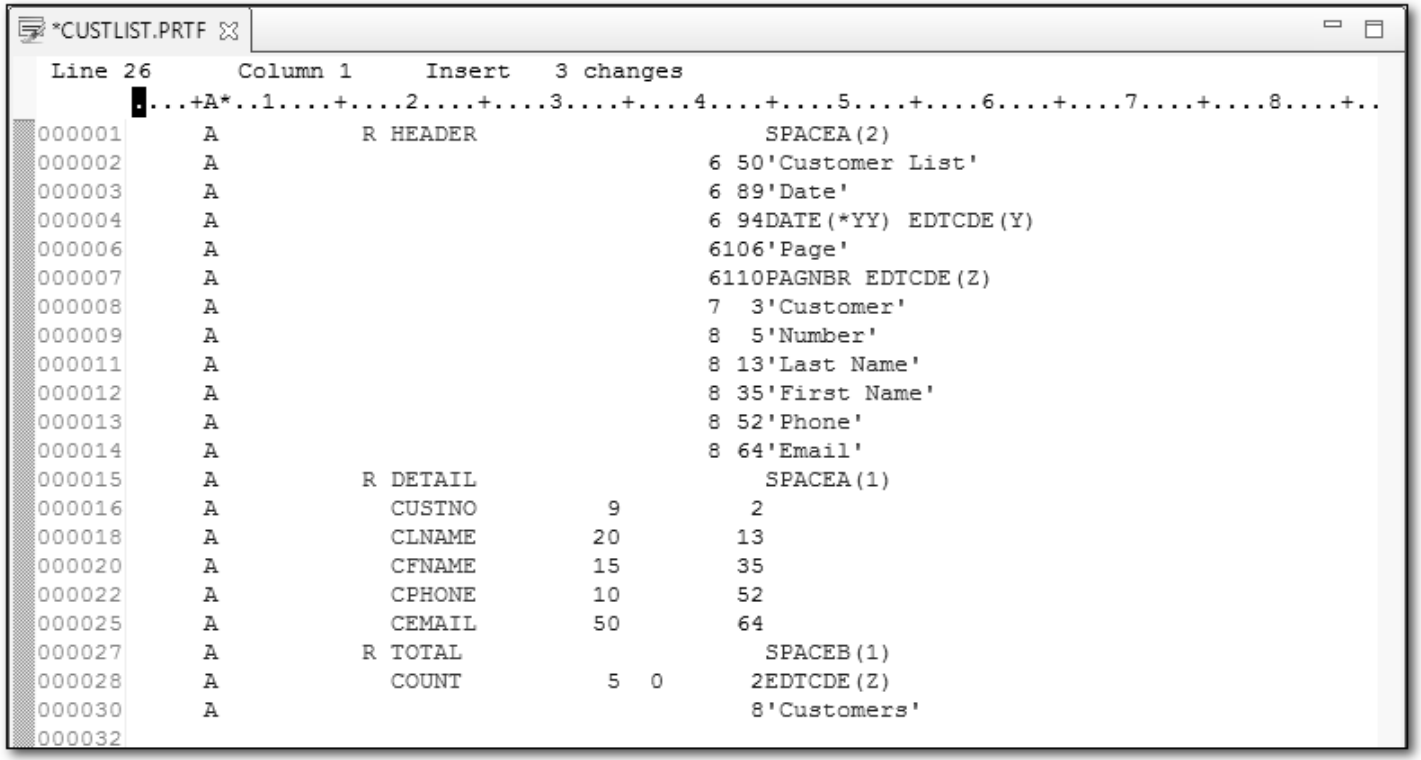
Externally Described Printer Files - Focus DDS for Printer Files

- Record formats include printed content
  - Fields, variables
  - Constants,literals



Externally Described Printer Files - Focus DDS for Printer Files

- DDS includes content position
  - Vertical line number, horizontal position
  - SPACEB, SPACEA, SKIPB, SKIPA keywords



Externally Described Printer Files - Focus DDS for Printer Files

- Keywords provide formatting information
  - DATE, PAGNBR
  - EDTCDE, EDTWRD

*CUSTLIST.PRTF

```
Line 26      Column 1     Insert   3 changes
      ....+A*..1....+....2....+....3....+....4....+....5....+....6....+....7....+....8....+..
000001      A          R HEADER                      SPACEA(2)
000002      A                                      6 50'Customer List'
000003      A                                      6 89'Date'
000004      A                                      6 94DATE(*YY) EDTCDE(Y)
000006      A                                      6106'Page'
000007      A                                      6110PAGNBR EDTCDE(Z)
000008      A                                      7  3'Customer'
000009      A                                      8  5'Number'
000011      A                                      8 13'Last Name'
000012      A                                      8 35'First Name'
000013      A                                      8 52'Phone'
000014      A                                      8 64'Email'
000015      A          R DETAIL                      SPACEA(1)
000016      A            CUSTNO         9          2
000018      A            CLNAME        20         13
000020      A            CFNAME        15         35
000022      A            CPHONE        10         52
000025      A            CEMAIL        50         64
000027      A          R TOTAL                       SPACEB(1)
000028      A            COUNT          5  0       2EDTCDE(Z)
000030      A                                       8'Customers'
000032
```

Edit Codes
DDS for Printer Files

- EDTCDE keyword indicates how to present numeric data
  - Thousands separators (commas)
  - Numeric sign
    - No sign shown
    - CR shown to right of number
    - Sign shown to right of number
    - Sign floats to left of number
  - Print zero balances

Edit Code	Positive	Negative	Zero
1	9,999,999.99	9,999,999.99	.00
2	9,999,999.99	9,999,999.99	
3	9999999.99	9999999.99	.00
4	9999999.99	9999999.99	
A	9,999,999.99	9,999,999.99CR	.00
B	9,999,999.99	9,999,999.99CR	
C	9999999.99	9999999.99CR	.00
D	9999999.99	9999999.99CR	
J	9,999,999.99	9,999,999.99-	.00
K	9,999,999.99	9,999,999.99-	
L	9999999.99	9999999.99-	.00
M	9999999.99	9999999.99-	
N	9,999,999.99	-9,999,999.99	.00
O	9,999,999.99	-9,999,999.99	
P	9999999.99	-9999999.99	.00
Q	9999999.99	-9999999.99	

Edit Codes
DDS for Printer Files

- Edit codes may include currency symbol or asterisk fill
  - E.g., EDTCDE(J '$') = $12,345.67
  - E.g., EDTCDE(J *) = **12,345.67
- EDTCDE(Z) provides simple zero suppression
  - No decimal or sign shown
  - EDTCDE(X) retains leading zeros
- Edit codes may insert slash separators into numbers used as dates
  - EDTCDE(Y)
    - 12252015 shown as 12/25/2015
    - 122515 shown as 12/25/15
  - EDTCDE(W)
    - 20151225 shown as 2015/12/25
    - 151225 shown as 15/12/25

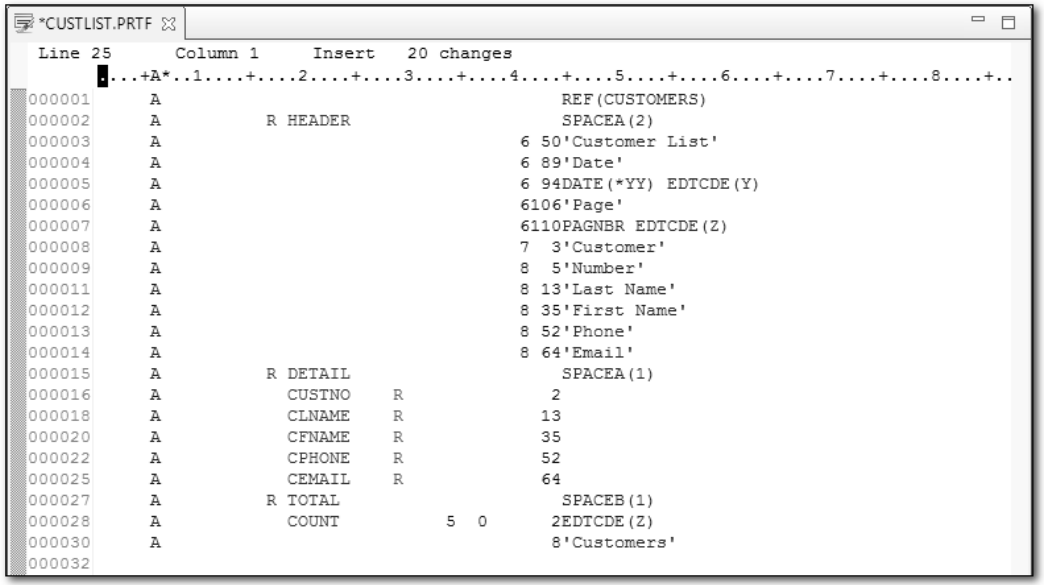
Edit Words
DDS for Printer Files

- EDTWRD keyword provides template for editing not supported by edit codes

EDTWRD(' - - ')	999-99-9999
EDTWRD(' & & ')	999 99 9999
EDTWRD('0( ) - ')	(999)999-9999
EDTWRD(' , **DOLLARS* *CENTS')	*9,999*DOLLARS*99*CENTS

Using a Field Reference File
DDS for Printer Files

- DDS field descriptions can inherit data attributes of fields in another externally described file
- REF field level keyword names field reference file
- R in position 29 specifies reference field
  - REFFLD keyword may name parent field, if different



Legacy Code: Program Described Files (more on this later)

- Record layout must be described in program source
  - Cannot use external definitions
- Fixed format Input and Output specifications describe layout
- May be appropriate when processing flat files

Legacy Code: Program Described Files

- File specification usually used to declare program described file
  - Or specify record length on Dcl-f instruction

```
Dcl-f Customers Disk(145);
Dcl-f Qprint    Printer(132) Oflind(*Inof);
```

```
FFilename++IPEASFRlen+LKlen+AIDevice+.Keywords+++++++++++++++++++++++++++++++

FCustomers IF   F  145        Disk
FQprint    O    F  132        Printer Oflind(*Inof)
```

Legacy Code: Program Described Files

- Input specifications describe record layout(s) for input file(s)
  - Must follow other declarations

```
IFilename++SqNORiPos1+NCCPos2+NCCPos3+NCC.....................................
I.....................Fmt+SPFrom+To+++DcField+++++++++L1M1FrPlMnZr......

ICustomers NS
I                                       1    9  Custno
I                                      10   24  Cfname
I                                      25   44  Clname
I                                      80   89  Cphone
I                                      90  139  Cemail
```

Legacy Code: Program Described Files

- Output specifications describe record layout(s) for output file(s)
  - Must follow calculations

```
OFilename++DF..N01N02N03Excnam++++B++A++Sb+Sa+.............................
O.............N01N02N03Field+++++++++YB.End++PConstant/editword/DTformat++

OQprint    E            Header          1  6
O                                          62 'Customer List'
O                                          92 'Date'
O                      *Date           Y   103
O                                         109 'Page'
O                      Page            Z   113

O          E           Detail           1
O                      Custno                10
O                      Clname                32
O                      Cfname                49
O                      Cphone                61
O                      Cemail               113
```